TCP File Transfer with SHA-256

High-Impact Visual Report for Demonstration and Evaluation

Bhargav Shekokar
Roll Number: B23CS1008

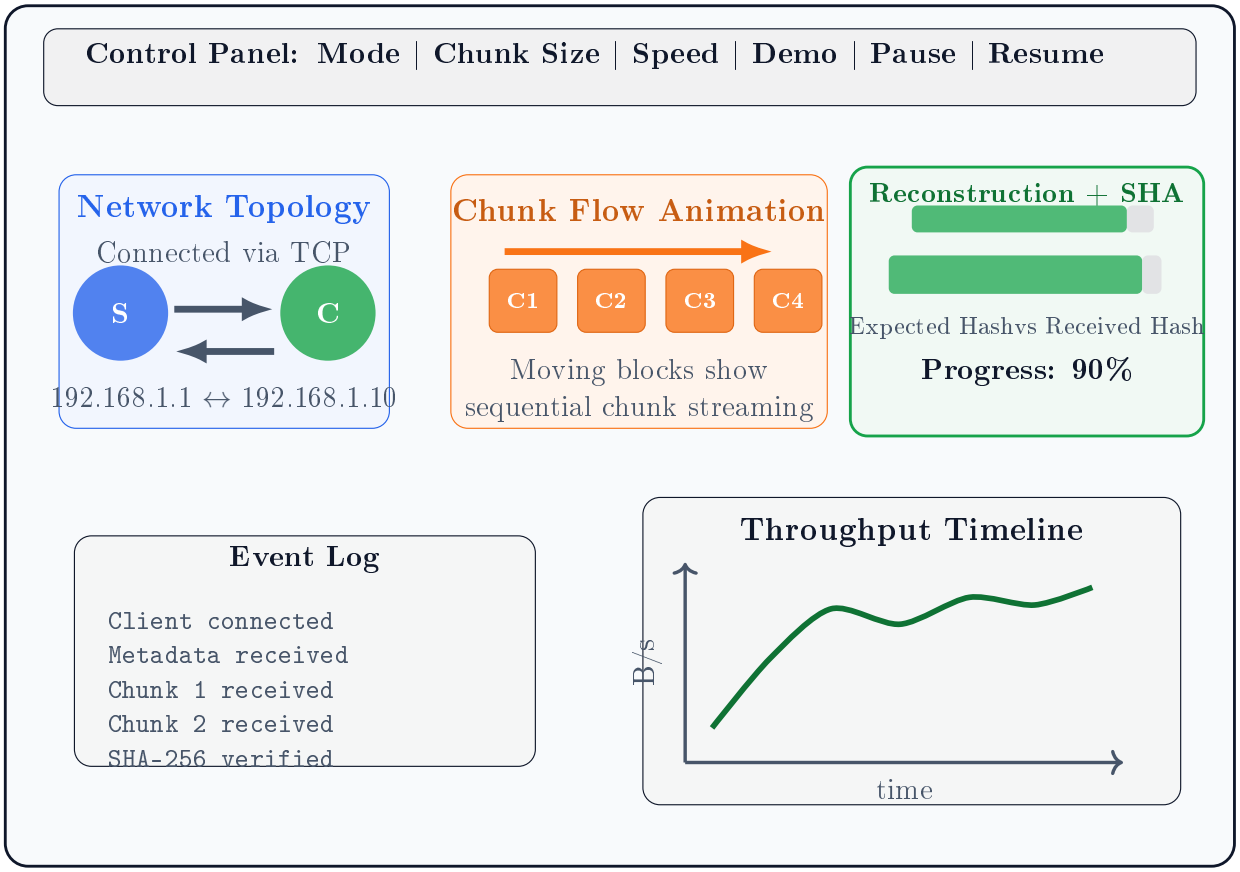
April 6, 2026

Mission Snapshot

This report is designed to be visually rich and presentation-ready. It explains the **correct transfer path**, highlights **probable failure points**, and gives you figures that are useful while implementing, demonstrating, and defending the project.

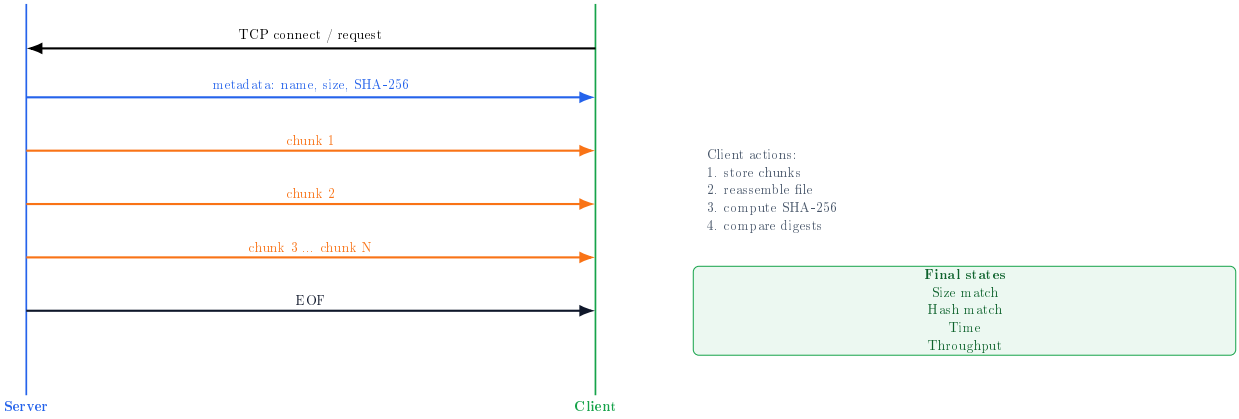
How to read this report: Each section pairs a diagram with a plain explanation so you can narrate what happens in a TCP file transfer. Use the dashboard to point out major roles, the protocol anatomy to discuss framing, the chunking comparison to motivate application-level design, and the failure gallery to show what breaks when assumptions are violated.

1. Big Picture Dashboard



2. End-to-End TCP Transfer Story

This swimlane is your quick script: connect, send metadata, stream ordered chunks, send EOF, then reconstruct and verify. When presenting, emphasize that TCP handles delivery but the application still controls framing and integrity checks.



3. Protocol Anatomy

The application-layer protocol rides on TCP. Highlight that metadata is a single JSON line, each chunk is length-prefixed with a sequence number, and EOF is explicit so the client knows when to stop reading.

Step 1: Metadata Line

```
{"filename":"file.bin","filesize":1048576,"sha256":"...","chunk_size":1024}
```

Step 2: Repeated Chunk Frames

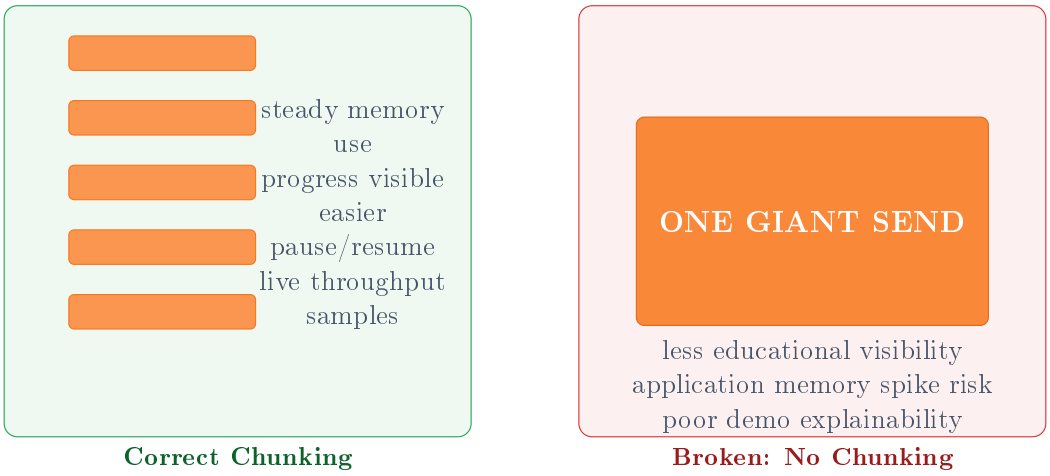
frame_size	seq_no	chunk_data

Step 3: End Marker

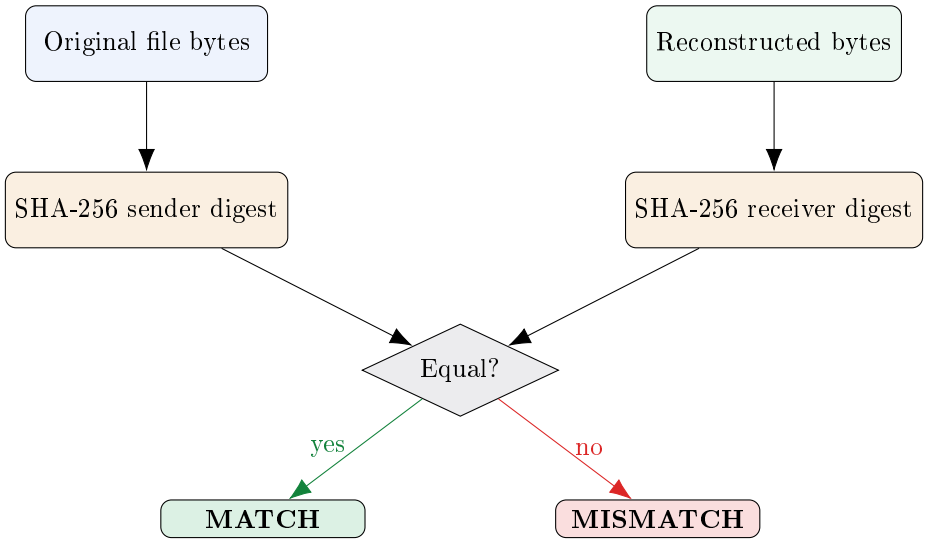
EOF

4. Why Chunking Matters

Chunking keeps memory steady and makes progress visible. The right pane shows how skipping chunking hides state and can stress memory. Mention that chunk size is tunable to explore throughput vs. CPU/UI overhead.



5. Integrity Verification Logic



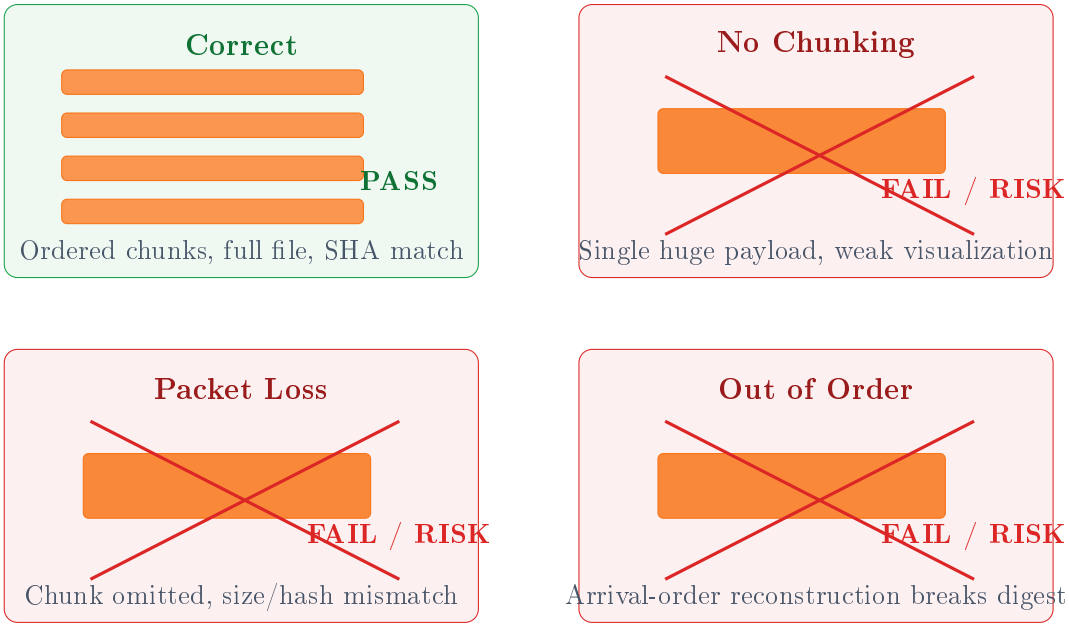
6. Probable Issues You Should Expect

These are the viva-ready talking points. Tie each issue back to what the GUI will visibly show during a demo.

Most Likely Problems During Demo / Viva / Testing

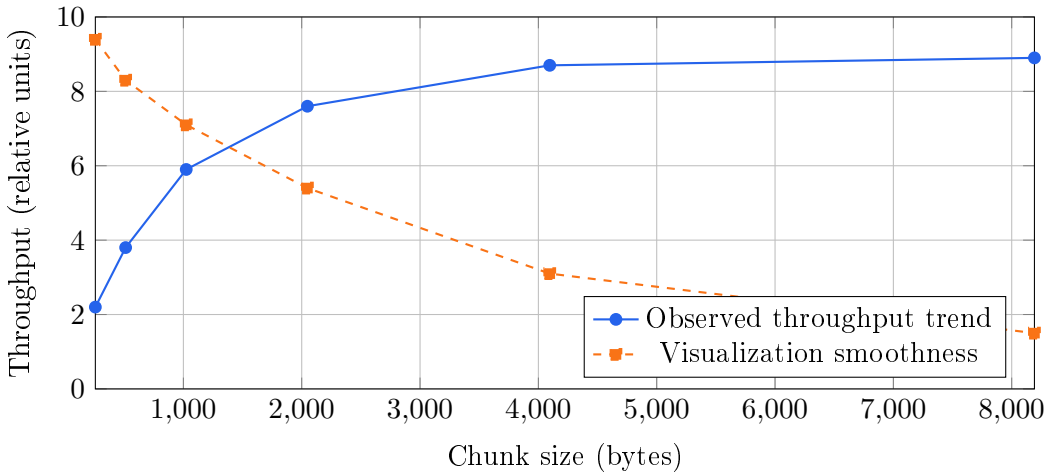
Issue	**Symptoms**	**Why it happens / what to show**
GUI freeze	buttons stop responding, animation stalls	networking or file I/O ran on the Tkinter main thread; explain use of `threading + queue + after()`
Incorrect EOF handling	client hangs or crashes near end of transfer	application does not distinguish frame header bytes from end marker cleanly
Wrong reconstruction order	file size may match but hash fails	chunks were appended by arrival order instead of sequence order; great viva question because TCP gives ordered bytes, but your app may still reorder logical frames incorrectly
Missing hash verification	transfer appears successful	students often assume TCP alone guarantees full file correctness; show why SHA-256 catches silent logical corruption
Huge chunk size	poor animation and bursty metrics	fewer updates, less pedagogical value, larger memory burst per receive
Tiny chunk size	lower throughput due to overhead	too many frames, too much Python and GUI overhead
Packet-loss simulation confusion	audience asks why TCP did not recover it	clarify that this mode simulates **application-layer omission**, not actual TCP segment loss
Port already in use	server cannot bind	old process still running or same port reused without cleanup
Firewall or multi-PC failure	client cannot connect	wrong IP, blocked port, or server bound only to localhost

7. Failure Mode Gallery



8. Throughput and Chunk-Size Tradeoff

Use this plot to justify your chosen default chunk size. Larger chunks reduce overhead but can smooth out the animation; smaller chunks improve instructional clarity but can lower throughput.



9. Recommended Talking Points for Your Task

What to Emphasize

- TCP gives reliable ordered byte delivery.
- Chunking improves control, visibility, and memory behavior.
- SHA-256 proves end-to-end integrity.
- Throughput must be measured, not assumed.
- Failure modes are educational, not just bugs.

Common Questions You May Get

- Why use SHA-256 if TCP is reliable?
- Why not send the whole file at once?
- What happens if a chunk is skipped?
- Can file size match but hash still fail?
- How do you keep the GUI responsive?

10. Implementation Notes Worth Mentioning

```
# GUI responsiveness pattern
network_thread -> queue.Queue() -> root.after(...) -> Tkinter widgets

# Verification
actual_hash = hashlib.sha256(reconstructed_bytes).hexdigest()
hash_match = (actual_hash == expected_hash)

# Throughput
throughput = bytes_received / elapsed_seconds
```

11. Final Conclusion

The strongest version of this project is not just "file transfer works." It is:

- **visually demonstrable**,
- **performance-aware**,
- **integrity-aware**,
- **able to explain failure**,
- and **easy to defend in an academic setting**.

That is why the report focuses on **chunk flow**, **reconstruction logic**, **hash comparison**, and **probable issues** rather than only showing code.